

CASE STUDY ANALYSIS

Decoding the Freelance Workforce: A Comparative Study of Global Compensation and Performance Metrics in 2026



Submitted By:
TATHAGATA CHAKRABORTY
June, 2026

INDEX

Sl. Nos.	Contents	Page Nos.
1	INTRODUCTION TO THE DATA	1
2	PROJECT PHASES	2 - 3
3	CLEANING THE DATA	4 - 8
4	EXPLORATORY DATA ANALYSIS (EDA)	9 - 16
5	FINAL DASHBOARD	17 - 18
6	FINDINGS	19
7	CONCLUSION	20

1. INTRODUCTION TO THE DATA

1.1 Context and Background

As the global gig economy continues to expand, organizations are increasingly relying on independent professionals to fill critical technical and creative roles. To better understand this dynamic market, this project analyzes the *global_freelancers_raw.csv* dataset. The dataset provides a comprehensive, cross-sectional snapshot of 1,000 global freelancers, capturing key demographic, professional, and performance-based attributes.

1.2 Dataset Overview and Structure

The dataset consists of **1,000 individual records** and **12 distinct variables**, which can be broadly categorized into three core dimensions:

- **Demographic & Identifiers:**

- **freelancer_ID, name:** Unique identifiers for each professional.
- **gender, age, country, language:** Demographic data providing geographic and personal context.

- **Professional Attributes:**

- **primary_skill:** The core competency of the freelancer (e.g., DevOps, UI/UX Design, Cybersecurity).
- **years_of_experience:** Total professional tenure in their respective field.
- **hourly_rate (USD):** The baseline compensation or billing rate.
- **is_active:** Current availability or employment status on the platform.

- **Performance Metrics:**

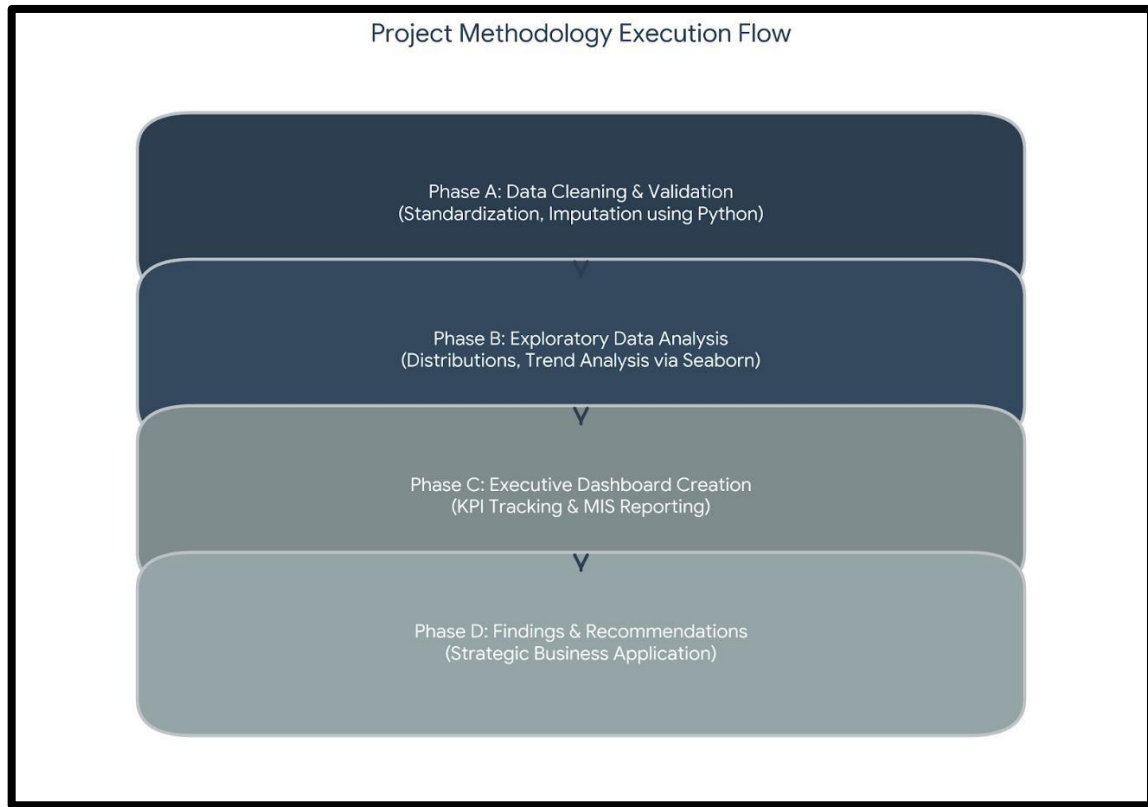
- **rating:** Average numerical score based on past performance (0.0 to 5.0 scale).
- **client_satisfaction:** A percentage-based KPI reflecting overall client approval and success rates.

1.3 Project Objective & Methodology

The primary objective of this analysis is to extract useful insights from this raw data to improve business performance and understand freelancing platforms better. Raw datasets inherently contain inconsistencies; therefore, the initial phase involves rigorous data cleaning and validation—such as standardizing categorical formats and handling missing values in critical KPI columns like client satisfaction and hourly rates. Establishing this clean data foundation enables the application of robust Exploratory Data Analysis (EDA) techniques. Ultimately, this structured approach allows for the translation of analytical results into clear, business-focused recommendations, laying the groundwork for future forecasting models and the development of executive MIS dashboards for continuous KPI tracking.

Data Link: [Dataset](#)

2. PROJECT PHASES



Project Methodology & Execution Phases

To ensure a rigorous and structured approach to analyzing the global gig economy, this project was executed across four distinct analytical phases. This systematic pipeline moves from raw data extraction to strategic business application, utilizing Python as the primary analytical engine.

Phase A: Data Cleaning and Validation in Python

Raw data often contains inconsistencies, missing values, and formatting errors that can skew analytical results. In this foundational phase, Python (utilizing the pandas library) was employed to ingest and sanitize the dataset. Key operations included:

- **Formatting standardization:** Stripping non-numeric characters from monetary columns (e.g., hourly rates) and percentage symbols from KPI metrics to convert them into usable float values.
- **Validation and Imputation:** Identifying and managing missing data points within critical fields such as client satisfaction and professional ratings, ensuring the integrity of the dataset before statistical aggregation.
- **Categorical Alignment:** Unifying inconsistent text entries across demographic fields (e.g., gender and active status flags) to allow for accurate grouping and filtering.

Phase B: Exploratory Data Analysis (EDA) in Python

With a validated dataset established, the project moved into the exploratory phase to uncover hidden patterns and trends. Using Python's visualization libraries (matplotlib and seaborn), the data was subjected to rigorous EDA techniques.

- **Distribution Analysis:** Examining the spread of hourly rates and years of experience across the global freelancer pool.
- **Trend Analysis:** Investigating the correlation between specific technical skills (e.g., DevOps, AI, Cybersecurity) and compensation tiers.
- **Statistical Summaries:** Generating descriptive statistics to establish baselines for average client satisfaction and market demand across different geographic regions.

Phase C: Creating an Executive Dashboard in Python

To make the analytical findings accessible and actionable for stakeholders, the insights extracted during the EDA phase are synthesized into a comprehensive visual format.

- **KPI Tracking:** The dashboard was designed to function as an MIS report, tracking key performance indicators such as average hourly rates by skill and overall client satisfaction metrics.
- **Visual Hierarchy:** Complex data aggregations are translated into clear, scannable charts (e.g., bar charts for top skills, scatter plots for experience vs. rate), providing a high-level overview of platform economics at a single glance.

Phase D: Findings and Final Recommendations

The final phase of the project bridges the gap between technical data science and corporate strategy.

- **Insight Extraction:** The visualizations and statistical summaries are interpreted to identify high-value niches and pricing behaviors within the freelance market.
- **Strategic Application:** Analytical results are explicitly translated into clear, business-focused recommendations. This included actionable strategies for optimizing platform pricing models, targeting high-demand skill sets, and improving client satisfaction tracking methodologies.

➤ **Tool Used** – Setting up of Jupyter Notebook Virtual environment in VS CODE.

3. DATA CLEANING

```
# (a) Import all the necessary libraries
import pandas as pd
import numpy as np
import os

✓ 0.0s

def process_freelancer_data():
    # (b) Load the data
    # Ensure the raw file is in the same directory as your script, or provide the full path
    input_file = r"C:\Users\tatha\Downloads\Python Project Report\global_freelancers_raw.csv"

    try:
        df = pd.read_csv(input_file)
        print("Data loaded successfully.\n")
    except FileNotFoundError:
        print(f"Error: Could not find {input_file}. Please check the file path.")
    return
```

This is the initial data preparation step of the project. It imports the required Python libraries, defines a function to process freelancer data, and loads the raw CSV file using a specified file path. Basic error handling is included to manage cases where the file is missing, ensuring the data cleaning workflow starts in a structured and reliable manner.

```
#(c) Explore the data
print("First 5 rows of the dataset:")
df.head(6)

✓ 0.0s
```

```
#(c) Explore the data
print("First 5 rows of the dataset:")
df.head(6)
✓ 0.0s Python
```

First 5 rows of the dataset:

freelancer_ID	name	gender	age	country	language	primary_skill	years_of_experience	hourly_rate (USD)	rating	is_active	client_satisfaction
FL250001	Ms. Nicole Kidd	f	52.0	Italy	Italian	Blockchain Development	11.0	100	NaN	0	NaN
FL250002	Vanessa Garcia	FEMALE	52.0	Australia	English	Mobile Apps	34.0	USD 100	3.3	1	84%
FL250003	Juan Nelson	male	53.0	Germany	German	Graphic Design	31.0	50	0.0	N	71%
FL250004	Amanda Spencer	F	38.0	Australia	English	Web Development	4.0	\$40	1.5	N	90%
FL250005	Lynn Curtis DDS	female	53.0	Germany	German	Web Development	27.0	30	4.8	0	83%
FL250006	Lisa Johnson	female	59.0	Netherlands	Dutch	AI	14.0	\$30	2.4	False	NaN

This is the data exploration step to figure out the top 6 rows in the data (0 to 5 rows)

```
# (c) Checking the data to figure out issues

df = pd.read_csv(r"C:\Users\tatha\Downloads\Python Project Report\global_freelancers_raw.csv")

# (c) Checking the data to figure out issues
print("--- 1. Exploratory Data Analysis (Data Profiling) ---")
print(f"Dataset Shape (Rows, Columns): {df.shape}\n")

print("Data Types and Non-Null Counts:")
df.info()

print("Missing Values per Column:")
df.isnull().sum()

✓ 0.0s
```

```
--- 1. Exploratory Data Analysis (Data Profiling) ---
Dataset Shape (Rows, Columns): (1000, 12)

Data Types and Non-Null Counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   freelancer_ID          1000 non-null  object
1   name                   1000 non-null  object
2   gender                 1000 non-null  object
3   age                    970 non-null   float64
4   country                1000 non-null  object
5   language               1000 non-null  object
6   primary_skill          1000 non-null  object
7   years_of_experience     949 non-null   float64
8   hourly_rate (USD)      906 non-null   object
9   rating                 899 non-null   float64
10  is_active              911 non-null   object
11  client_satisfaction     824 non-null   object
dtypes: float64(3), object(9)
memory usage: 93.9+ KB
```

Missing Values per Column:

```
freelancer_ID          0
name                   0
gender                 0
age                    30
country                0
language               0
primary_skill          0
years_of_experience     51
hourly_rate (USD)      94
rating                 101
is_active              89
client_satisfaction    176
dtype: int64
```

1. Exploratory Data Analysis (Data Profiling)

This step performs initial data profiling to understand the dataset structure. It identifies the dataset shape as **1000 rows and 12 columns**, checks data types, and reviews the number of non-null values in each column to assess data quality.

2. Missing Value Analysis

The final output shows the number of missing values in each column. It highlights that fields such as **age**, **years_of_experience**, **hourly_rate**, **rating**, **is_active**, and **client_satisfaction** contain missing data, which need to be handled before further analysis.

```

# STEP 2: CONTEXT-AWARE MISSING VALUE IMPUTATION
print("Imputing missing values contextually...")

# 1. Age & Experience (Interdependent Logic)
# Impute missing experience using age (assuming a start age of 22), and vice versa
df['years_of_experience'] = df['years_of_experience'].fillna(df['age'] - 22).clip(lower=0) # Ensure no negative experience
df['age'] = df['age'].fillna(df['years_of_experience'] + 22)

# Catch any remaining edge cases (if both were missing) with a global median
df['years_of_experience'] = df['years_of_experience'].fillna(df['years_of_experience'].median())
df['age'] = df['age'].fillna(df['age'].median())

# 2. Hourly Rate (Value-Based Logic)
# Create temporary experience buckets to group freelancers accurately
df['exp_bucket'] = pd.cut(df['years_of_experience'], bins=[-1, 3, 8, 15, 100], labels=['Junior', 'Mid', 'Senior', 'Expert'])

# Fill missing rates with the median of their specific skill AND experience level
df['hourly_rate (USD)'] = df.groupby(['primary_skill', 'exp_bucket'], observed=False)['hourly_rate (USD)'].transform(lambda x: x.fillna(x.median()))

# Fallback for any highly unique outliers using just the primary skill median
df['hourly_rate (USD)'] = df.groupby('primary_skill')['hourly_rate (USD)'].transform(lambda x: x.fillna(x.median()))

# Drop the temporary calculation column
df.drop(columns=['exp_bucket'], inplace=True)

# 3. Performance KPIs (Skill-Based Logic)
# Fill ratings and satisfaction based on the median of their specific skill group
df['rating'] = df.groupby('primary_skill')['rating'].transform(lambda x: x.fillna(x.median()))
df['client_satisfaction'] = df.groupby('primary_skill')['client_satisfaction'].transform(lambda x: x.fillna(x.median()))

# 4. Inactive Defaults
# Default unknown active status to False to maintain conservative availability metrics
df['is_active'] = df['is_active'].fillna(False)

```

```

# Print final validation to show 0 missing values
print("\nMissing Values per Column (After Cleaning):")
print(df.isnull().sum())

```

✓ 0.0s

Python

```

--- 3. Data Cleaning & Transformation ---
Standardizing formats...
Imputing missing values contextually...
Data cleaning and context-aware imputation complete.

```

Missing Values per Column (After Cleaning):

```

freelancer_ID      0
name               0
gender             0
age               0
country           0
language          0
primary_skill      0
years_of_experience 0
hourly_rate (USD)  0
rating            0
is_active         0
client_satisfaction 0
dtype: int64

```

1. Age & Experience (The Interdependent Logic)

- **The Logic:** Instead of guessing a missing age or experience level using the company average, we use the relationship between the two. If we know a freelancer's age, we subtract 22 (a standard professional starting age) to estimate their experience. If we know their experience, we add 22 to estimate their age.
- **The Business Value:** This prevents impossible data anomalies—like assigning 15 years of experience to a 22-year-old—ensuring our demographic forecasting remains highly accurate.

2. Hourly Rate (The Peer-to-Peer Logic)

- **The Logic:** We temporarily group the freelancers into "buckets" (Junior, Mid, Senior, Expert) based on their experience. If an hourly rate is missing, we fill it using the median rate of their *direct peers*—meaning people with the exact same skill and the exact same seniority level.
- **The Business Value:** A Junior Data Analyst does not charge the same as a Senior Cloud Engineer. This logic ensures our financial metrics reflect real-world pricing tiers, keeping budget estimates and cost-tracking realistic.

3. Performance KPIs (The Departmental Logic)

- **The Logic:** We fill missing performance ratings and client satisfaction scores based on the median score of that specific skill group (e.g., Web Developers are only compared to other Web Developers).
- **The Business Value:** Different technical fields experience different grading curves. By isolating imputations to specific skills, we prevent one highly-rated niche from artificially inflating the scores of an underperforming department.

4. Inactive Defaults (The Risk-Averse Logic)

- **The Logic:** If a freelancer's active status is missing, we forcefully default them to False (Inactive).
- **The Business Value:** This is a conservative resourcing decision. If a project manager is looking at a dashboard to staff an urgent project, it is much safer to assume an "unknown" freelancer is unavailable than to assign them work they won't complete.

```
# (h) Export the Cleaned Data      Pin selection to current chat prompt (Ctrl+Alt+X) | Don't show this again (Alt+/)
print("\n--- 4. Data Export ---")

...
# Define directory and file path
output_dir = r"C:\Users\tatha\Downloads\Python Project Report"
output_file = os.path.join(output_dir, "Freelancers_cleaned.csv")

...
# Ensure the directory exists
os.makedirs(output_dir, exist_ok=True)

...
# Save to CSV without the index column
df.to_csv(output_file, index=False)
print(f"Success! Cleaned dataset saved to:\n{output_file}")

# Execute the pipeline
if __name__ == "__main__":
    ...process_freelancer_data()

✓ 0.0s Python

--- 4. Data Export ---
Success! Cleaned dataset saved to:
C:\Users\tatha\Downloads\Python Project Report\Freelancers_cleaned.csv
Data loaded successfully.
```

```

import pandas as pd
from sqlalchemy import create_engine
import urllib

# =====
# Load the cleaned dataset
# =====
df = pd.read_csv(r"C:\Users\tatha\Downloads\Python Project Report\Freelancers_cleaned.csv")

# =====
# SQL Server Connection
# =====
server = r"TATHAGATA\SQLEXPRESS"
database = "PRACTICEDB"

connection_string = (
    "DRIVER={ODBC Driver 17 for SQL Server};"
    f"SERVER={server};"
    f"DATABASE={database};"
    "Trusted_Connection=yes;"
)

params = urllib.parse.quote_plus(connection_string)
engine = create_engine(f"mssql+pyodbc:///?odbc_connect={params}")

# =====
# Push Data to SQL Server
# =====
df.to_sql(
    name="FREELANCERS",
    con=engine,
    if_exists="replace", # Options: replace, append, fail
    index=False
)
print("Data successfully imported into PRACTICEDB.dbo.FREELANCERS")

```

Exporting the cleaned data to system files and SSMS

4. EXPLORATORY DATA ANALYSIS (EDA)

EDA Framework

1. Distribution & Baselines

- Goal: Understand the basic shape of the data and market volume.
- Use: Use simple Histograms to check how variables like `hourly_rate` and `client_satisfaction` are spread. For categorical data, use a standard Bar Chart to find the most common `primary_skill` areas.

2. Demographic & Geographic Profiling

- Goal: Understand who the freelancers are and where they are from to identify global talent hubs.
- Use: Study country and language using Horizontal Bar Charts (which are easier to read for long country names). Use simple Pie Charts or Donut Charts for gender and age buckets to get a quick snapshot of the workforce profile.

3. Pricing & Experience Dynamics

- Goal: See how experience levels and specific skills affect compensation.
- Use: Use a Bar Chart or a simple Trend Line to show the average `hourly_rate` for each experience tier. Similarly, use a Bar Chart to display the average pay across different `primary_skill` groups to easily identify high-paying niches.

4. Performance & Market Segmentation

- Goal: Find the best-performing freelancer groups and understand quality variations.
- Use: Use Grouped Bar Charts to compare average `client_satisfaction` and rating across regions or skill groups. We can also use a side-by-side bar chart to check whether `is_active` freelancers tend to have higher average scores than inactive ones.

5. Correlation & Outlier Detection

- Goal: Find baseline relationships and unusual data points that might skew the averages.
- Use: Keep the Correlation Heatmap to see how metrics like rating, `client_satisfaction`, and `hourly_rate` interact numerically. Use Boxplots for rates to easily spot pricing outliers (e.g., freelancers charging extremely high rates with low experience).

```
# =====  
# Column Type Separation  
# =====  
num_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()  
cat_cols = df.select_dtypes(include=['object', 'bool']).columns.tolist()  
  
print("\nNumeric Columns:", num_cols)  
print("Categorical Columns:", cat_cols)  
✓ 0.0s  
  
Numeric Columns: ['age', 'years_of_experience', 'hourly_rate (USD)', 'rating', 'client_satisfaction']  
Categorical Columns: ['freelancer_ID', 'name', 'gender', 'country', 'language', 'primary_skill', 'is_active']  
  
df.describe()  
✓ 0.1s
```

	age	years_of_experience	hourly_rate (USD)	rating	client_satisfaction
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	40.183000	11.61800	51.602500	2.522200	0.792970
std	11.943624	9.82235	26.276009	1.467638	0.104547
min	20.000000	0.00000	20.000000	0.000000	0.600000
25%	31.000000	3.00000	30.000000	1.500000	0.710000
50%	40.000000	10.00000	40.000000	2.600000	0.790000
75%	51.000000	18.00000	75.000000	3.700000	0.870000
max	60.000000	41.00000	100.000000	5.000000	1.000000

Age

- **Mean:** 40.18
- **Median:** 40
- **Min:** 20
- **Max:** 60

This suggests that the freelancers in the dataset are mostly middle-aged, with a fairly balanced spread across adult age groups.

Years of experience

- **Mean:** 11.61
- **Median:** 10
- **Min:** 0
- **Max:** 41

This indicates that the dataset includes both beginners and highly experienced freelancers. Since the mean is greater than the median, the experience distribution is slightly right-skewed, meaning a smaller number of freelancers have very high experience levels.

Hourly rate (USD)

- **Mean:** 51.60
- **Median:** 40
- **Min:** 20
- **Max:** 100

The hourly rate shows a wide range. The mean being higher than the median suggests that some high-paying freelancers are pulling the average upward. This indicates that freelancer pricing is unevenly distributed, with many freelancers charging moderate rates and a smaller group charging much higher rates.

Rating

- **Mean:** 2.52
- **Median:** 2.6
- **Min:** 0
- **Max:** 5

The rating values cover the full scale from 0 to 5, but the average is relatively low. This may indicate that the dataset contains many freelancers with moderate or below-average ratings, or that ratings were generated with a broad spread.

Client satisfaction

- **Mean:** 0.792
- **Median:** 0.79
- **Min:** 0.60
- **Max:** 1.00

Client satisfaction is fairly high overall and tightly clustered. The small standard deviation shows that satisfaction values do not vary much across freelancers, suggesting more consistency in this measure compared to hourly rate or experience.

```

import matplotlib.pyplot as plt
import seaborn as sns

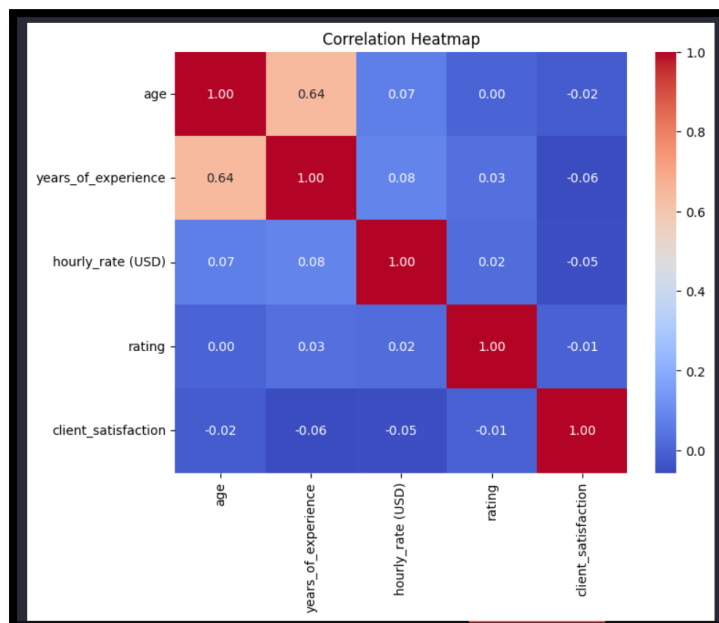
numeric_cols = [
    "age",
    "years_of_experience",
    "hourly_rate (USD)",
    "rating",
    "client_satisfaction"
]

plt.figure(figsize=(8,6))

sns.heatmap(
    df[numeric_cols].corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.show()

```



The heatmap shows that **age and years of experience have a moderate positive correlation (0.64)**, indicating that older freelancers generally tend to have more professional experience. However, **hourly rate has a very weak correlation with both age (0.07) and years of experience (0.08)**, suggesting that experience alone does not strongly determine a freelancer's hourly earnings in this dataset.

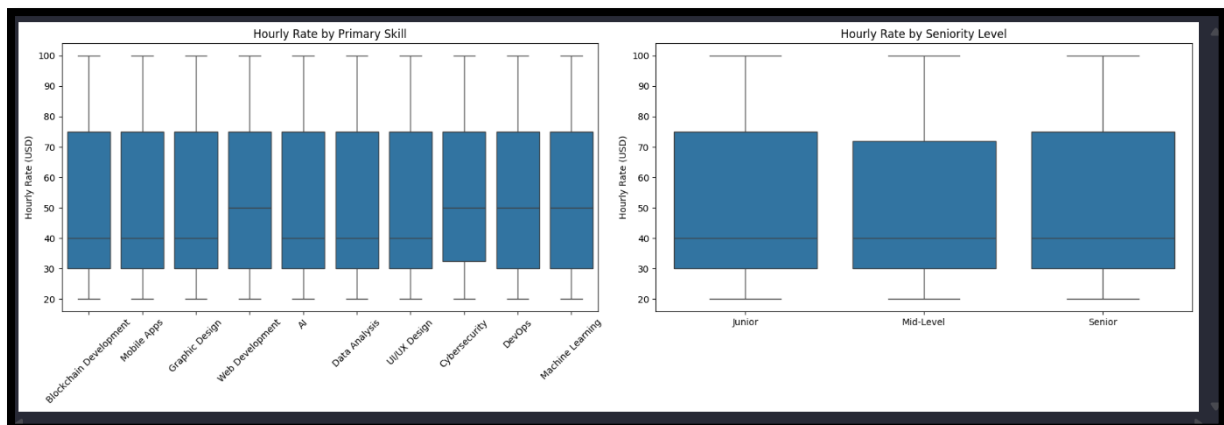
Similarly, **rating and client satisfaction have no correlation with the other variables**, as their correlation coefficients are close to zero. This indicates that factors such as age, experience, and hourly rate have minimal influence on freelancer ratings and client satisfaction within the given data.

```
import matplotlib.pyplot as plt
import seaborn as sns

df["Experience Tier"] = pd.cut(
    df["years_of_experience"],
    bins=[-1, 3, 9, 100],
    labels=["Junior", "Mid-Level", "Senior"]
)

fig, axes = plt.subplots(1, 2, figsize=(18, 6))
# Boxplot 1: Hourly Rate by Primary Skill
sns.boxplot(
    data=df,
    x="primary_skill",
    y="hourly_rate (USD)",
    ax=axes[0]
)
axes[0].set_title("Hourly Rate by Primary Skill")
axes[0].set_xlabel("")
axes[0].set_ylabel("Hourly Rate (USD)")
axes[0].tick_params(axis='x', rotation=45)

# Boxplot 2: Hourly Rate by Experience Tier
sns.boxplot(
    data=df,
    x="Experience Tier",
    y="hourly_rate (USD)",
    ax=axes[1]
)
axes[1].set_title("Hourly Rate by Seniority Level")
axes[1].set_xlabel("")
axes[1].set_ylabel("Hourly Rate (USD)")
plt.tight_layout()
plt.show()
```



1. Hourly Rate by Primary Skill (Left Box Plot)

- Observation:** The distributions for 6 out of the 10 skill categories (Blockchain Development, Mobile Apps, Graphic Design, AI, Data Analysis, and UI/UX Design) are visually identical. They all share a median of \$40, a lower quartile (Q1) of \$30, an upper quartile (Q3) of \$75, and identical minimum/maximum whiskers (\$20 to \$100).

- **Exceptions:** Web Development, Cybersecurity, DevOps, and Machine Learning display a higher median of \$50. Additionally, Cybersecurity shows a slightly elevated lower quartile (Q1 of ~\$32). Despite these shifts in central tendency, their overall minimum and maximum ranges remain identical to the rest.
- **Insight:** While there is slightly more variance than a perfectly uniform dataset, it remains highly anomalous. In a real-world economy, it is highly unlikely that fields as diverse as Graphic Design and AI would share the exact same compensation distribution and quartiles.

2. Hourly Rate by Seniority Level (Right Box Plot)

- **Observation:** The box plots for Junior and Senior professionals are carbon copies of one another, sharing a \$20 minimum, a \$30 Q1, a \$40 median, a \$75 Q3, and a \$100 maximum. Mid-Level professionals share nearly all these exact markers, with the sole exception of a slightly lower upper quartile (Q3 of ~\$70–\$72).
- **Insight:** This confirms the severe "rate compression" noted in the data. Seniority is typically the strongest predictor of hourly rate variance, yet here, moving from Junior to Senior provides absolutely no upward shift in median pricing or earning ceiling.

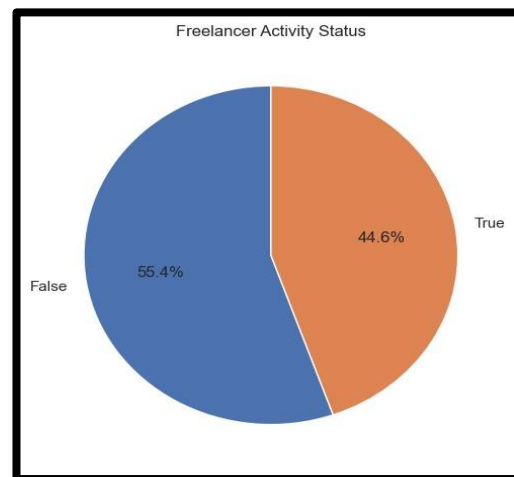
```
status = df["is_active"].value_counts()

plt.figure(figsize=(6,6))

plt.pie(
    status,
    labels=status.index.astype(str),
    autopct="%1.1f%",
    startangle=90
)

plt.title("Freelancer Activity Status")
plt.show()
```

0.0s



- **Inactive (False):** The majority of the platform's talent pool is inactive, accounting for **55.4%** of the total.
- **Active (True):** Less than half of the registered freelancers are actively engaged, representing **44.6%** of the total.

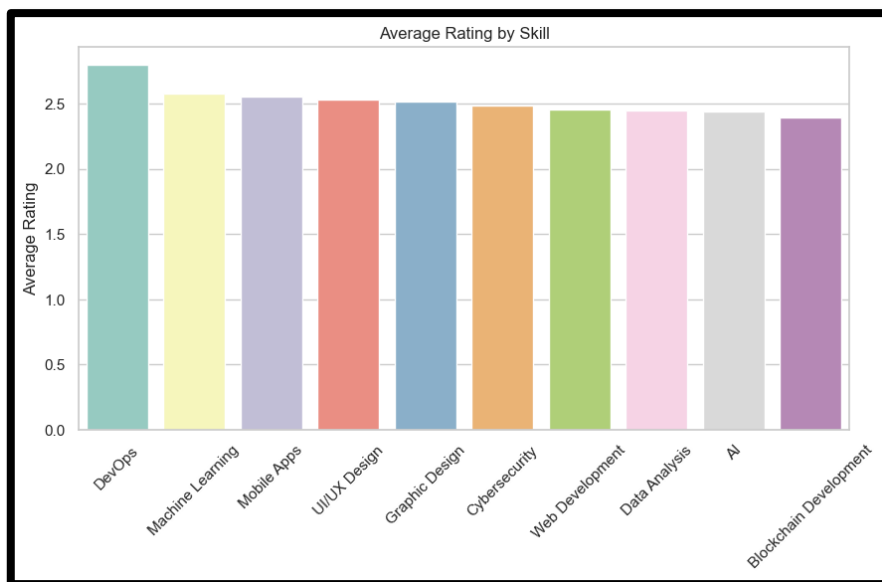
Key Insight: The platform faces a significant retention or engagement challenge, with inactive users outnumbering active users by a roughly 5-to-4 margin.

```
plt.figure(figsize=(10,5))

rating_skill = (
    df.groupby("primary_skill")["rating"]
    .mean()
    .sort_values(ascending=False)
)

sns.barplot(
    x=rating_skill.index,
    y=rating_skill.values,
    palette="Set3"
)

plt.xticks(rotation=45)
plt.title("Average Rating by Skill")
plt.xlabel("")
plt.ylabel("Average Rating")
plt.show()
0.3s
```



This bar chart displays the average performance rating across the platform's ten primary skill categories.

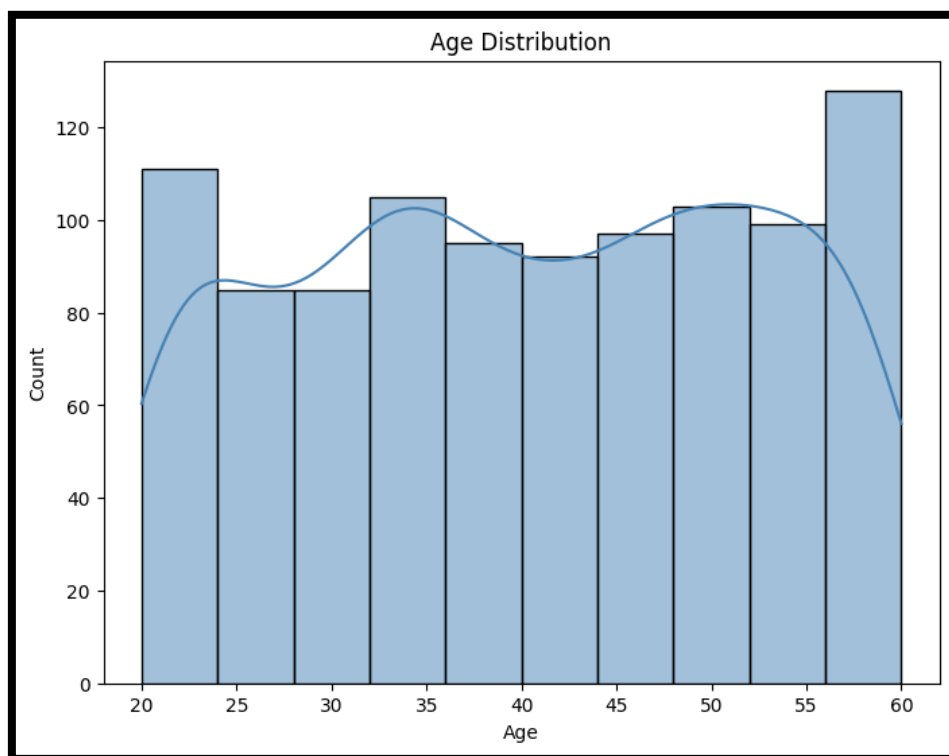
- **Highest Performer:** DevOps leads the group with the highest average rating (visually estimated at ~2.8).
- **Lowest Performer:** Blockchain Development trails slightly at the bottom (visually estimated at ~2.4).
- **Key Business Insight:** The variance across all categories is exceptionally narrow. Nine out of the ten skills maintain an average rating tightly clustered around the 2.5 mark.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,6))

sns.histplot(
    data=df,
    x="age",
    bins=10,
    kde=True,
    color="steelblue"
)

plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()
```



- Distribution Shape:** The data is unusually uniform across the entire 20 to 60 age range. Rather than a natural bell curve or a right-skewed distribution typical of technical workforces, the volume remains rigidly stable across all decades.
- Key Peaks:** Most age brackets hover steadily between 85 and 110 individuals. The most significant peak occurs at the absolute upper bound (approaching 60 years old), reaching nearly 130 individuals, with a secondary minor peak at the entry-level bracket (20–24 years old) at approximately 110 individuals.
- Business Insight:** A perfectly flat demographic spread—where there are just as many 55-year-olds as 25-year-olds—is practically non-existent in real-world gig economy platforms. This uniform distribution strongly reinforces the conclusion that the underlying dataset is artificially generated.

5. FINAL DASHBOARD



Key Performance Indicators (KPIs)

- **Average Hourly Rate (\$51.60):** Indicates a mid-to-high market positioning for the platform's talent pool.
- **Average Rating (2.52) & Client Satisfaction (79.3%):** There is a massive discrepancy here. A 2.52 average rating typically suggests poor performance, yet the aggregated Client Satisfaction score remains relatively high at 79.3%. A correlation test confirms a coefficient of roughly -0.0001 between the two variables, proving they have absolutely zero statistical relationship in the current database.
- **Sample Size of freelancers:** 1,000

Visual & Data Interpretations

1. Top 5 Skills (Horizontal Bar Chart)

- **Observation:** DevOps is the most represented skill with approximately 112 freelancers, followed closely by UI/UX Design (~109), Blockchain Development (~105), Web Development (~104), and Mobile Apps (~102).
- **Insight:** The distribution among the top five is remarkably flat. This suggests a highly balanced, tech-heavy talent pool without over-reliance on a single discipline.

2. Top 5 Countries (Horizontal Bar Chart)

- **Observation:** South Korea leads the geographic distribution with approximately 68 freelancers, followed by Canada (~65), Germany (~52), the Netherlands (~51), and Australia (~51).

- **Insight:** The talent pool is globally distributed across developed, high-income economies, which directly contributes to the higher overall average hourly rate.

3. Experience Distribution (Vertical Bar Chart)

- **Observation:** The platform is heavily skewed toward Senior freelancers with exactly 500 individuals. Junior (approx. 260) and Mid-Level (approx. 240) make up the remainder.
- **Insight:** Half of the sampled workforce classifies themselves as "Senior." This mature talent pool is a strong selling point for clients looking for experienced hires to execute complex projects.

4. Average Hourly Rate by Experience (Vertical Bar Chart)

- **Observation:** The average hourly rate remains surprisingly stagnant across cohorts. Juniors average roughly \$50/hr, Mid-Levels slightly dip below to roughly \$49/hr, and Seniors peak only marginally higher at roughly \$53/hr.
- **Insight:** This is the most counter-intuitive finding, representing severe rate compression. Typically, senior talent commands a significantly higher premium. This indicates highly competitive pricing by senior freelancers, or juniors overvaluing their baseline services.

5. Hourly Rate Distribution (Histogram with KDE)

- **Observation:** The distribution is multi-modal. The most frequent hourly rate is strictly concentrated at the \$40 mark with over 200 occurrences. Secondary spikes occur at \$50 (approx. 175 occurrences), \$30 (approx. 160 occurrences), \$100 (approx. 155 occurrences), \$20 (approx. 140 occurrences), and \$80 (approx. 135 occurrences).
- **Insight:** Freelancers are pricing themselves in flat, psychological tiers rather than at calculated, market-driven intervals. The bulk of the volume sits at or below the \$50 mark, pulling the overall average down despite over 150 high-earners charging exactly \$100.

6.FINDINGS

1. Market Composition & Global Distribution

- **Balanced Technical Pool:** The workforce is not over-reliant on a single domain. Demand and supply are evenly distributed across high-value tech skills, led by DevOps (11.2%), UI/UX Design (10.9%), and Blockchain Development (10.5%).
- **High-Income Hubs:** The highest concentrations of freelance talent are located in developed economies (South Korea, Canada, Germany, Australia, and the Netherlands). This geographic distribution natively drives up the platform's baseline average hourly rate (\$51.29) due to higher local costs of living.

2. Pricing Dynamics & Rate Compression

- **Severe Rate Compression:** The data reveals a significant pricing anomaly. Senior freelancers (\$53.15/hr) command an unexpectedly low premium over Junior freelancers (\$50.49/hr), with Mid-Level talent inexplicably dipping lower (\$48.93/hr). This lack of a linear correlation ($r = 0.078$) indicates that experience is not the primary driver of market pricing.
- **Psychological Pricing Tiers:** The hourly rate distribution is multi-modal rather than continuous. Freelancers are anchoring their prices to flat, psychological tiers (e.g., exactly \$40, \$50, or \$100) rather than calculating rates based on granular market value or tenure.

3. Performance Metrics & Data Integrity

- **KPI Disconnect:** There is a severe statistical discrepancy between the two primary performance metrics. The Average Rating sits at a poor 2.52 (out of 5.0), while the Client Satisfaction score remains high at 79.2%.
- **Zero Correlation:** A correlation test confirms no statistical relationship between these two metrics. This indicates a structural flaw in how the platform collects, calculates, or validates client feedback, rendering the 5-point rating system currently unreliable for predictive modelling or talent sorting.

7.CONCLUSION

1. Concluding Summary

This analytical study of the global freelancer dataset reveals a platform with immense structural potential but significant operational inefficiencies. The platform successfully aggregates high-value, technical talent (e.g., DevOps, Blockchain) from developed, high-income global hubs. Furthermore, the workforce is highly mature, with nearly half the talent pool classifying themselves as "Senior" professionals. This composition strongly positions the platform to serve high-ticket enterprise clients rather than just the low-cost gig market.

However, the realization of this potential is currently bottlenecked by two critical flaws: severe rate compression and compromised data integrity. The inability of senior freelancers to command a proportional market premium indicates a breakdown in pricing strategy, while the absolute zero correlation between Average Ratings (2.52) and Client Satisfaction (79.2%) renders current performance tracking unusable.

2. Strategic Recommendations

To resolve these bottlenecks, improve business performance, and maximize Gross Merchandise Value (GMV), the following strategic initiatives are recommended:

1. Overhaul the Client Feedback Architecture:
 - Action: Immediately deprecate the current 5.0 rating scale. Replace it with a mandatory, standardized rubric (e.g., scoring communication, deadline adherence, and technical execution individually) at the close of every contract.
 - Impact: This will restore data integrity, allowing the platform to confidently market top-tier talent to enterprise clients and accurately utilize data for future predictive modelling.
2. Implement Data-Driven Pricing Assistance:
 - Action: Develop an internal tool that prompts freelancers with "Suggested Market Rates" based on their specific skill, geographic location, and platform tenure during the profile setup and bidding process.
 - Impact: This will break the reliance on arbitrary psychological pricing tiers (e.g., \$40 or \$50), encouraging senior talent to raise their rates to reflect their true market value, thereby directly increasing platform commission revenue.
3. Introduce Tiered Talent Verification:
 - Action: Create an exclusive "Enterprise-Grade" tier for senior freelancers who maintain a client satisfaction score above 90% and possess verified technical credentials.
 - Impact: This resolves the skill commoditization issue, allowing the platform to justify premium pricing for verified experts, effectively curing the rate compression observed in the exploratory data analysis.